

Laboratorio 1: Redes de Computadores

Sistema Híbrido de Registro, Autenticación, Presencia y Mensajería (HTTP, TCP y UDP)

Profesor: Jorge Díaz

Ayudantes: Cristóbal Brandau, Juan Cucurella

Agosto 2026

1 Objetivos del laboratorio

- Implementar una arquitectura cliente-servidor utilizando sockets TCP y UDP en los lenguajes **Python** y **Go**.
- Construir un servicio de registro de usuarios basado en el protocolo HTTP y persistencia en disco mediante archivos CSV.
- Diseñar e implementar un protocolo de autenticación, mensajería y retransmisión concurrente (*broadcast*) en tiempo real sobre sockets TCP y UDP.

2 Introducción

Un socket corresponde a la interfaz que comunica las capas de aplicación y de transporte en el modelo de redes. Este mecanismo permite establecer flujos de datos bidireccionales entre procesos independientes, ejecutándose ya sea en una misma máquina anfitriona o en nodos remotos distribuidos en una red de área local o Internet.

En la capa de transporte coexisten dos protocolos fundamentales con características operativas contrastantes:

- **TCP (*Transmission Control Protocol*)**: Es un protocolo orientado a la conexión que provee entrega de datos confiable, ordenada y libre de errores mediante acuse de recibo (*ACK*), retransmisión de segmentos perdidos y control de flujo. Resulta indispensable en escenarios críticos como autenticación, transacciones y mensajería garantizada.
- **UDP (*User Datagram Protocol*)**: Es un protocolo no orientado a la conexión, ligero y sin sobrecarga de sincronización previa. Aunque no garantiza la entrega ni el orden secuencial de los datagramas, su mínima latencia lo convierte en la opción estándar para transmisiones periódicas de telemetría, transmisiones de voz/video y señales de vida (*heartbeats*).

Los sistemas distribuidos modernos y las plataformas de chat en tiempo real combinan estratégicamente ambos protocolos: emplean canales HTTP/TCP para operaciones deterministas de registro, autenticación e intercambio confiable de mensajes con retransmisión a múltiples usuarios, y delegan en canales UDP la comprobación periódica de conectividad de los clientes conectados.

En este laboratorio, su objetivo es implementar un sistema modular de comunicación de tres capas (HTTP, TCP y UDP) que interoperará de forma cruzada entre los lenguajes **Python** y **Go**, aplicando conceptos clave de concurrencia, sincronización de estado, delimitación de tramas, recepción asíncrona de mensajes y persistencia en archivos de datos.

3 Laboratorio 1

3.1 Enunciado

Tras los incidentes y tensiones operativas en los puestos fronterizos del Borde Exterior, el Alto Mando ha decidido implementar un nuevo protocolo de comunicaciones encubierto para sus destacamentos de avanzada.

Para impedir que agentes no autorizados intercepten o corrompan los canales de comunicación, el mando central exige un sistema riguroso de tres fases: los agentes deben registrarse formalmente a través de un canal HTTP seguro, autenticar su identidad estableciendo un enlace punto a punto confiable vía TCP para obtener un token criptográfico temporal, y mantener activo un pulso constante de señal de radiofrecuencia (canal UDP) que certifique su operatividad en terreno.

Si un agente pierde emisión de su pulso por más de un minuto, el servidor central asumirá que la unidad ha sido comprometida o desconectada de emergencia, revocando su token de sesión de forma irreversible, cerrando su canal de comunicación y notificando su desconexión a la red.

Su equipo de ingenieros de comunicaciones ha sido designado para construir la totalidad del sistema (clientes y servidores), garantizando que los mensajes enviados por cualquier miembro sean propagados en tiempo real hacia los demás agentes conectados y que el sistema sea completamente compatible e interoperable entre distintas plataformas y lenguajes.

3.2 Explicación general

El sistema está estructurado en tres componentes coordinados que deben operar de manera sincronizada:

1. **Canal de Registro (HTTP):** Un cliente HTTP envía los datos de registro (`username` y `password`) hacia un servidor HTTP. Este último procesa la solicitud y almacena las credenciales en un archivo estructurado denominado `usuarios.csv`.
2. **Canal de Autenticación y Enlace TCP:** El usuario se conecta a través de un cliente TCP al servidor TCP para iniciar sesión. Si las credenciales coinciden con las registradas en `usuarios.csv`, el servidor genera un token de sesión alfanumérico único con un tiempo máximo de vida (TTL) de 10 minutos, registra la sesión activa en el archivo `sesiones.csv`, asocia el socket TCP activo al usuario autenticado, abre/asigna un socket UDP en el servidor, y responde al cliente por TCP entregándole el token de sesión y el número de puerto UDP asignado.
3. **Canal de Presencia / Heartbeat (UDP):** Una vez recibido el token y el puerto, el cliente dispone de una ventana de gracia de máximo **30 segundos** para inicializar un subproceso o hilo en segundo plano que transmitirá periódicamente datagramas UDP con el token de sesión cada **3 segundos** hacia el servidor UDP. El servidor actualiza el registro de actividad de dicho token.
4. **Expiración y Revocación de Sesión:** Si transcurre **1 minuto (60 segundos)** continuos sin que el servidor UDP reciba un *heartbeat* válido asociado a un token activo, o si no se recibe el primer latido dentro de los 30 segundos iniciales, el servidor revoca la sesión, elimina/marca el token en `sesiones.csv` y procede a cerrar el socket TCP del usuario infractor.
5. **Mensajería Concurrente y Retransmisión (TCP):** Mientras el token de sesión permanezca activo y verificado:
 - El cliente TCP puede enviar un mensaje al servidor.
 - El servidor valida la sesión y registra el mensaje en el historial persistente `historial.csv`.
 - El servidor retransmite (*broadcast*) inmediatamente el mensaje hacia **todos los demás clientes TCP que mantengan una sesión activa**, garantizando que todos los usuarios reciban y visualicen la interacción en tiempo real en sus respectivas pantallas.

3.2.1 Asignación de Lenguajes por Paridad de Grupo

Para asegurar el dominio práctico de sockets en múltiples entornos y evaluar la interoperabilidad estricta de protocolos de red, la implementación de los componentes se distribuirá según el número de grupo asignado en Aula:

Asignación	Lado Servidor (HTTP, TCP, UDP)	Lado Cliente (HTTP, TCP, UDP)
Grupos de Número IMPAR	Python (versión 3.10+)	Go (versión 1.20+)
Grupos de Número PAR	Go (versión 1.20+)	Python (versión 3.10+)

Table 1: Distribución de lenguajes de programación según número de grupo.

Nota: Ambos lados deben apegarse estrictamente a los formatos de mensaje, delimitadores y códigos de respuesta acordados para que un cliente en Go pueda comunicarse de manera transparente con un servidor en Python y viceversa.

3.3 Pre-laboratorio

Antes de comenzar el desarrollo de la experiencia, cada integrante del equipo debe:

- Leer detalladamente el presente enunciado y las reglas de entrega.
- Tener instalado en su entorno de trabajo **Python 3.10** o superior y el compilador de **Go 1.20** o superior.
- Verificar el funcionamiento básico de sockets en ambos lenguajes mediante pruebas locales de enlace (*loopback* / *127.0.0.1*).
- Familiarizarse con herramientas de prueba y diagnóstico de red en terminal como **netcat** (**nc**), **curl** o **tcpdump**.

3.4 Componentes a implementar

3.4.1 Componente 1 — Servicio HTTP de Registro

Permite la incorporación de nuevos usuarios al sistema antes de intentar cualquier conexión en capa de transporte.

- **Servidor HTTP:** Escucha en un puerto configurable (por defecto 8080). Debe exponer el endpoint **POST /register**, el cual recibe las credenciales del usuario (**username** y **password**) ya sea en formato JSON o codificado por formulario. Además, debe exponer el endpoint **GET /history**, el cual presente el historial de mensajes, almacenado por el servidor, al usuario.
- **Persistencia en CSV:** Valida que el nombre de usuario no se encuentre registrado previamente. En caso exitoso, almacena el registro en el archivo **usuarios.csv** con la estructura:

`username,password,fecha_registro`

- **Respuestas HTTP:** Debe retornar código de estado **201 Created** ante un registro exitoso, **409 Conflict** si el usuario ya existe, y **400 Bad Request** si el cuerpo de la solicitud no contiene los parámetros requeridos.
- **Cliente HTTP:** Programa o módulo cliente que construye la solicitud HTTP **POST** con los datos del nuevo usuario, la envía al servidor e informa en consola el resultado de la operación. ***Nota: Toda la interacción realizada por el cliente HTTP debe ser realizada a través de sockets TCP.***

3.4.2 Componente 2 — Servidor TCP de Autenticación, Mensajería y Retransmisión

Constituye el núcleo del sistema y debe gestionar múltiples clientes conectados simultáneamente de forma concurrente (mediante hilos/bloqueos en Python o *goroutines* en Go). Además, debe mantener en memoria una lista o mapa de los sockets de clientes autenticados y activos para permitir la retransmisión de mensajes.

- **Autenticación (LOGIN):** Recibe la solicitud de inicio de sesión con formato de comando estructurado, por ejemplo:

```
LOGIN <username> <password>\n
```

El servidor consulta el archivo `usuarios.csv`. Si las credenciales coinciden:

1. Genera un identificador de sesión alfanumérico único (*Session Token*) con un TTL de 10 minutos.
2. Registra la sesión activa en el archivo `sesiones.csv` con la estructura:

```
token,username,timestamp_creacion,timestamp_ultimo_heartbeat,estado
```

3. Registra el socket TCP del cliente en su lista de sesiones activas conectadas.
4. Responde al cliente vía TCP informando el éxito, el token generado y el puerto UDP asignado:
OK <token> <puerto_udp>\n

Si las credenciales son incorrectas o el usuario no existe, responde con `ERROR_INVALID_CREDENTIALS\n`.

- **Envío y Retransmisión de Mensajes (MSG / BROADCAST):** Recibe solicitudes de publicación de mensajes bajo el formato:

```
MSG <token> <contenido_del_mensaje>\n
```

El servidor valida que el token exista en `sesiones.csv`, que su estado sea `ACTIVO`, que no hayan transcurrido más de 10 minutos desde su creación y que no haya caído en timeout por inactividad UDP (> 60 s).

- **Si la sesión es válida:**

1. Registra el mensaje en `historial.csv` con la estructura:

```
timestamp,username,mensaje
```

2. Responde al remitente con el acuse de recibo: `ACK\n`.

3. **Retransmisión (Broadcast):** Envía de inmediato a través de sus sockets TCP activos el mensaje a todos los demás clientes con sesión válida, utilizando un formato claro como:

```
INCOMING <usuario_emisor> <contenido_del_mensaje>\n
```

- **Si la sesión expiró o es inválida:** Responde al cliente con `ERROR_SESSION_EXPIRED\n` o `ERROR_INVALID_TOKEN\n`, rechaza el registro del mensaje y no realiza retransmisión.

- **Cierre de Conexión (LOGOUT o Expiración):** Al desconectarse un cliente o expirar su sesión, el servidor retira su socket de la lista activa, libera los descriptores asociados y elimina o invalida el token de sesión correspondiente.

3.4.3 Componente 3 — Canal UDP de Heartbeat y Monitor de Expiración

Garantiza la supervisión continua del estado de conexión de los clientes mediante datagramas no bloqueantes.

- **Servidor UDP (Receptor de Heartbeat):** Escucha datagramas entrantes con el formato:
`HEARTBEAT <token>`

Al recibir un paquete, verifica la existencia del token y actualiza de inmediato el campo `timestamp_ultimo_heartbeat` correspondiente a esa sesión.

- **Monitor de Inactividad (Watchdog):** Rutina en segundo plano en el servidor que evalúa periódicamente el estado de las sesiones activas en `sesiones.csv`:

- Si un cliente inicia sesión vía TCP pero no envía su primer datagrama UDP en un plazo de **30 segundos**, la sesión queda anulada.

- Si transcurre más de **1 minuto (60 segundos)** desde el último heartbeat recibido de un token activo, el servidor anula la sesión eliminando o invalidando el token, cierra el socket TCP del cliente correspondiente y lo remueve de la lista de difusión.
- Si se alcanzan los **10 minutos** de vida máxima del token desde su emisión, la sesión se anula automáticamente.

3.4.4 Componente 4 — Cliente Integrado (Recepción Concurrente)

Es la aplicación que interactúa con todos los servicios provistos:

- Permite registrar un usuario nuevo a través del cliente HTTP.
- Se conecta al servidor TCP y ejecuta el comando de inicio de sesión (LOGIN).
- Al recibir el token de sesión y el puerto UDP, lanza dos rutinas o hilos concurrentes en segundo plano:
 1. **Hilo de Heartbeat (UDP):** Despacha datagramas UDP HEARTBEAT <token> cada **3 segundos** de forma ininterrumpida hacia el puerto UDP informado por el servidor.
 2. **Hilo Receptor TCP (Lectura de Broadcasts):** Escucha continuamente en el socket TCP las respuestas del servidor y los mensajes entrantes (INCOMING <usuario> <mensaje>) emitidos por otros usuarios, mostrándolos de forma inmediata por consola.
- En el hilo principal de la consola, permite al usuario escribir y enviar mensajes mediante MSG, sin que la espera de escritura bloquee la visualización de mensajes entrantes.

3.5 Experiencia a realizar

El grupo de trabajo deberá ejecutar y evidenciar en conjunto el flujo operacional completo:

1. Iniciar el servidor (HTTP, TCP y UDP) en una terminal principal.
2. Ejecutar dos instancias de clientes en terminales separadas (**Cliente A** y **Cliente B**).
3. Realizar el registro de ambos usuarios mediante la interfaz/cliente HTTP y verificar la correcta escritura de credenciales en `usuarios.csv`.
4. Conectar ambos clientes al servidor TCP e iniciar sesión. Verificar en el servidor la emisión de tokens, la escritura en `sesiones.csv` y la llegada regular de los latidos UDP (cada 3 segundos) de ambos clientes.
5. **Prueba de Mensajería y Broadcast:** El Cliente A envía un mensaje a través de TCP. Verificar que:
 - El servidor TCP valida el token, guarda el mensaje en `historial.csv` y responde con ACK al Cliente A.
 - El Cliente B recibe automáticamente por su socket TCP el mensaje entrante y lo muestra en su terminal sin intervención del usuario.
6. **Prueba de Timeout por Inactividad (Watchdog):** Suspender o cancelar el hilo de envío UDP de uno de los clientes (o simular corte de heartbeat).
7. Dejar transcurrir más de **1 minuto (60 segundos)** y comprobar que el servidor revoca la sesión en `sesiones.csv` y cierra su canal TCP.
8. Intentar enviar un mensaje desde el cliente revocado y verificar que el servidor rechaza la solicitud retornando error de sesión expirada.
9. **Toda la interacción anterior debe ser registrada en un video continuo y fluido.**

3.5.1 Requerimientos mínimos

- Respetar estrictamente la asignación de lenguajes según la paridad del número de grupo (Grupos impares: Servidor en Python / Cliente en Go; Grupos pares: Servidor en Go / Cliente en Python).
- Todos los sockets y mensajes transmitidos en la red deben manejar codificación de caracteres en **UTF-8**.
- Los comandos TCP deben utilizar un delimitador de fin de mensaje claro (por ejemplo, salto de línea `\n`) para evitar problemas de fragmentación o lecturas bloqueantes en los sockets.
- Todas las consultas realizadas por el cliente HTTP deben ser realizadas utilizando sockets TCP, bajo ningún caso deben utilizarse clientes HTTP de alto nivel.
- La lectura y escritura sobre los archivos CSV (`usuarios.csv`, `sesiones.csv`, `historial.csv`) debe implementar mecanismos de exclusión mutua (*mutex* o bloqueos de concurrencia) para evitar condiciones de carrera y corrupción de datos ante accesos simultáneos.
- El cliente TCP debe ser totalmente concurrente: no debe bloquear la recepción e impresión de mensajes entrantes de otros usuarios mientras el usuario redacta en la consola.
- El servidor debe ser capaz de atender a múltiples clientes conectados y difundir los mensajes en tiempo real.
- El laboratorio debe ser capaz de correr completamente en al menos una máquina de manera local.
- El video de demostración debe tener una duración máxima de **2 a 3 minutos**, mostrando con claridad los terminales de ejecución (servidor y clientes interactuando entre sí), la inspección de los archivos CSV y la respuesta del sistema ante la prueba de timeout.
- El código fuente entregado debe estar debidamente estructurado, comentado y acompañado de un archivo `README.md` descriptivo.

4 Librerías permitidas

Para la realización del laboratorio se permite exclusivamente el uso de las bibliotecas estándar de ambos lenguajes:

Python

- `socket` (implementación de sockets TCP y UDP en capa de transporte).
- `threading` o `asyncio` (concurrencia, hilos de escucha y procesos en segundo plano).
- `http.server` y `urllib.request` (manejo del servicio y cliente HTTP).
- `csv` (lectura y persistencia de datos tabulares).
- `time`, `datetime`, `uuid`, `os`, `sys`.

Go

- Paquete `net` (`net.Listen`, `net.Dial`, `net.ListenUDP`, etc.).
- Paquete `net/http` (servidor y peticiones HTTP estándar).
- Paquetes `sync`, `time`, `os`, `fmt`, `bufio`, `strings`, `encoding/csv`.

*Queda **estrictamente prohibido** el uso de frameworks web de alto nivel externos (tales como Flask, FastAPI, Django, Gin, Fiber, Gorilla Mux, Requests en Python, etc.). El objetivo pedagógico es comprender la construcción de servicios a partir de las primitivas de red y librerías base del lenguaje.*

5 Reglas de entrega

- El laboratorio debe realizarse en grupos de **3 integrantes**, formalmente inscritos en la plataforma Aula.
- La fecha límite de entrega es el día **5 de Septiembre de 2026** a las 23:59 hrs.
- La entrega debe realizarse exclusivamente a través de Aula mediante un único archivo comprimido en formato **.zip**, con el siguiente formato de nombre:
L1-Grupo[Nº Grupo].zip (Ejemplo: **L1-Grupo03.zip**).
Solo se aceptarán archivos .zip. Cualquier otro formato (.rar, .7z, .tar.gz) será calificado como no entregado.
- El archivo comprimido debe incluir la totalidad del código fuente de los componentes (servidores y clientes), los archivos CSV base generados, el enlace o archivo del video demostrativo y un archivo **README.md**.
- El **README.md** debe especificar obligatoriamente:
 - Nombre completo y rol USM de los integrantes del grupo.
 - Instrucciones detalladas de compilación, ejecución y puertos por defecto.
 - Especificación y documentación formal de los comandos y respuestas de los protocolos TCP y UDP implementados.

La falta de alguna de estas secciones podría conllevar descuentos de **5 a 15 puntos** dependiendo de la complejidad que introduzca no contar con la información solicitada.

- **Penalizaciones:**
 - La ausencia del README, de la documentación del protocolo o del video de funcionamiento resultará en una penalización de **25 puntos** por cada elemento faltante.
 - Cada hora de retraso en la entrega implicará un descuento de **15 puntos**.
 - Errores en el formato de nombre del archivo de entrega conllevan una penalización de **15 puntos**.
 - Errores en los roles USM de los integrantes conllevan un descuento de **15 puntos**.
 - Faltas graves de ortografía en el informe o README conllevarán una penalización de entre **5 y 15 puntos**.
- Cualquier sospecha fundamentada de copia, plagio entre grupos o uso excesivo de asistentes generativos de código será calificada con **nota 0**, pudiendo citarse al grupo a una comisión de defensa oral del código entregado.